

SBA301 – INTEGRATE SINGLE PAGE APPLICATION WITH SPRING BOOT

HƯỚNG DẪN SINH VIÊN THỰC HIỆN

ASSIGNMENT 01 – WORKING WITH REACTJS APPLICATION

FUNewsManagementSystem • Bài tính điểm • Hướng dẫn triển khai và chuẩn bị đánh giá theo rubric

Nguyên tắc trọng tâm: Đây là tài liệu hướng dẫn để sinh viên tự phân tích, tự thiết kế, tự triển khai, tự kiểm thử và tự giải thích code.

Thông tin	Nội dung
Assignment	Assignment 01 – Working with ReactJS application
Bối cảnh	FUNewsManagementSystem – frontend quản trị bằng ReactJS
CLO	CLO1, CLO2
Hình thức	Bài có tính điểm; có demo/interview/verification
Điểm rubric	10.0 điểm / 8 tiêu chí
Tên project	StudentName_ClassCode
Tài liệu được phép	Slide, tài liệu chính thức, ví dụ đã học, AI có kiểm soát

Mục tiêu cuối: sản phẩm chạy được + sinh viên chứng minh được năng lực thật

CÁCH SỬ DỤNG TÀI LIỆU

Tài liệu này không phải đáp án. Hãy dùng nó như một bản đồ triển khai và một checklist chuẩn bị đánh giá. Mỗi milestone chỉ mô tả mục tiêu, quyết định kỹ thuật, bằng chứng cần có và câu hỏi mà sinh viên phải tự trả lời.

Giai đoạn	Việc sinh viên cần làm	Bằng chứng nên có
1. Understand	Đọc đề, lập Requirement Traceability Matrix, xác định màn hình/entity/constraint.	Bảng yêu cầu, component sketch, data sketch.
2. Design	Thiết kế layout, component tree, state ownership, navigation, CRUD flow.	Sơ đồ chữ/ảnh, design decisions.
3. Build	Triển khai theo milestone, commit theo chức năng, không viết toàn bộ một lần.	Git history, chức năng chạy từng phần.
4. Verify	Test normal/negative/edge case, sửa lỗi có log.	Test matrix, ảnh/log/debug notes.
5. Explain	Tự giải thích code, lệnh, state flow, event flow.	Interview rehearsal checklist.
6. Transfer	Thực hành một yêu cầu thay đổi nhỏ không dựa vào code mẫu.	Patch nhỏ + test mới.
7. Defend	Demo, trả lời interview, sửa lỗi/đổi yêu cầu khi được hỏi.	Bộ hồ sơ đánh giá hoàn chỉnh.

AI usage: AI có thể hỗ trợ giải thích, review, tạo test case, gợi ý debug và cải tiến. AI không thay thế việc sinh viên phải hiểu code và có thể sửa code trực tiếp khi đánh giá.

MỤC LỤC NỘI DUNG

- 0. Thông tin Assignment và chính sách đánh giá
- 1. Tóm tắt yêu cầu và phạm vi thực hiện
- 2. Hướng dẫn phân tích yêu cầu – Problem Understanding
- 3. Hướng dẫn thiết kế giải pháp – Solution Design & Reasoning
- 4. Hướng dẫn triển khai theo milestone – Implementation
- 5. Hướng dẫn kiểm thử – Testing & Verification
- 6. Hướng dẫn debug – Debugging & Problem Solving
- 7. Hướng dẫn chuẩn bị giải thích code – Code Explanation
- 8. Hướng dẫn xử lý yêu cầu thay đổi – Modification / Transfer Task
- 9. Hướng dẫn sử dụng AI có kiểm soát
- 10. Rubric đánh giá 10 điểm
- 11. Checklist trước khi nộp và trước buổi đánh giá
- 12. Phụ lục – Interview, lệnh và quy luật thực thi

0. THÔNG TIN ASSIGNMENT VÀ CHÍNH SÁCH ĐÁNH GIÁ

Mục	Yêu cầu
Tên bài	Assignment 01 – Working with ReactJS application
Hệ thống	FUNewsManagementSystem
Frontend	ReactJS + Vite + HTML/CSS template có sẵn
Core scope	Login; Header; Dashboard/Category/News/Users/Settings; CRUD + Search; dialog cho Create/Update; confirmation khi Delete
Mock authentication	Admin / Admin → admin page
Data	Có thể dùng JS array, localStorage, JSON hoặc mock service; không bắt buộc Spring Boot/database thật
Role tham khảo	Admin = 1; Staff = 2
Status tham khảo	News/Category: active = 1, inactive = 0
Project name	StudentName_ClassCode

0.1 Sản phẩm cần nộp

- Source code ReactJS có thể cài dependency và chạy lại.
- README.md: mô tả, cách cài/chạy, tài khoản test, chức năng, giới hạn, nguồn tham khảo/AI usage nếu có.
- Ảnh chụp các chức năng chính hoặc video demo nếu giảng viên yêu cầu.
- Test matrix hoặc bảng chứng kiểm thử theo yêu cầu của lớp.
- AI usage log nếu giảng viên yêu cầu.
- Link GitHub nếu quy trình lớp sử dụng Git/GitHub.

0.2 Chính sách hoàn thành đúng hạn

Lưu ý: Sinh viên cần nộp bài đúng hạn; Sau thời hạn quy định edunext sẽ không gia hạn nộp bài.

0.3 Chính sách trung thực học thuật và AI

- Sinh viên chịu trách nhiệm với toàn bộ code nộp và phải giải thích được code của mình.
- Không sao chép nguyên vẹn source của sinh viên khác hoặc source trọn bộ từ AI.
- Được tham khảo slide, tài liệu chính thức, ví dụ trên lớp và AI assistant theo quy định.
- **Nếu dùng AI: ghi prompt/mục đích, phần đã áp dụng, cách kiểm chứng và phần đã chỉnh sửa.**
- Nếu code chạy nhưng sinh viên không giải thích được hoặc không thể thay đổi một phần nhỏ, rubric Code Explanation/Modification/AI Usage sẽ phản ánh đúng năng lực.

0.4 Quy trình buổi đánh giá – mô hình khuyến nghị

Pha	Sinh viên thực hiện	Giảng viên quan sát/chấm
A. Identity & Run	Mở đúng project, chạy bằng lệnh chuẩn, đăng nhập và demo nhanh.	Tính xác thực project; môi trường; project chạy.
B. Requirement Demo	Demo login, layout/menu, CRUD/Search/dialog/confirmation.	Problem Understanding + Implementation.
C. Code Interview	Mở các file do giảng viên chỉ định và giải thích luồng.	Code Explanation + Design Reasoning.
D. Verification/Debug	Chạy một test/edge case hoặc phân tích một lỗi được đưa ra.	Testing + Debugging.
E. Transfer Task	Thực hiện hoặc mô tả chi tiết một thay đổi nhỏ.	Modification/Transfer.
F. AI Evidence	Trình bày AI log nếu có và cách đã kiểm chứng.	AI Usage & Engineering Practice.

Chuẩn bị đúng: Không học thuộc câu trả lời. Hãy có khả năng mở đúng file, chỉ đúng state/function, mô tả trước–trong–sau sự kiện và chứng minh bằng chạy chương trình.

1. TÓM TẮT YÊU CẦU VÀ PHẠM VI THỰC HIỆN

1.1 Bài Assignment yêu cầu sinh viên xây dựng gì?

Xây dựng frontend quản trị cho FUNewsManagementSystem bằng ReactJS, dựa trên HTML/CSS template có sẵn. Ứng dụng phải có login, master/admin layout và các khu vực quản lý Dashboard, Category, News, Users, Settings. Category, News và Account/User phải có CRUD + Search; Create/Update dùng popup dialog; Delete có confirmation.

1.2 Requirement Traceability Matrix tối thiểu

ID	Yêu cầu	Bằng chứng trên UI/code	Test tối thiểu
R01	Project ReactJS + Vite, tên đúng quy ước	package.json, folder name	npm run dev
R02	Login page	Login component/page	Empty, wrong, Admin/Admin
R03	Admin/Admin vào admin page	Auth state + navigation/conditional render	Đúng/sai credential
R04	Logo tạo bằng AI	Asset + Header	Asset hiển thị
R05	Header + master/admin layout	Header/Layout components	Các page nằm trong layout
R06	Menu Dashboard/Category/News/Users/Settings	Sidebar/Menu	Chuyển đúng nội dung
R07	Category CRUD + Search	Category management	Create/Read/Update/Delete/Search
R08	News CRUD + Search	News management	Create/Read/Update/Delete/Search
R09	User/Account CRUD + Search	User management	Create/Read/Update/Delete/Search
R10	Create/Update bằng popup dialog	Modal/Dialog + form state	Mở đúng mode, submit, close
R11	Delete confirmation	Confirm dialog/prompt	Cancel không xóa; confirm mới xóa
R12	Data model hợp lý	Mock data/service/localStorage	Role/status/category relation

1.3 Phạm vi không bắt buộc

- Không bắt buộc Spring Boot backend thật.
- Không bắt buộc database thật.
- Không bắt buộc JWT/authentication production.
- Không bắt buộc deployment production.
- Không bắt buộc chức năng ngoài đề nếu chúng không giúp chứng minh rubric.

Ưu tiên: Hoàn thành đúng và giải thích được core requirement trước khi thêm dark mode, chart, pagination hoặc thư viện nâng cao.

2. HƯỚNG DẪN PHÂN TÍCH YÊU CẦU – PROBLEM UNDERSTANDING

Mục tiêu rubric: sinh viên chứng minh rằng mình hiểu bài toán trước khi code. Một project chạy được nhưng thiếu constraint hoặc không biết input/output sẽ không đạt điểm tối đa.

2.1 Xác định màn hình và trách nhiệm

Màn hình/khu vực	Trách nhiệm tối thiểu	Câu hỏi phải trả lời
Login	Nhập credential; validate; login Admin/Admin; error khi sai.	Auth state lưu ở đâu? Sau login điều gì thay đổi?

Màn hình/khu vực	Trách nhiệm tối thiểu	Câu hỏi phải trả lời
Dashboard	Tổng quan; có thể thống kê đơn giản.	Dữ liệu nào có ý nghĩa để tổng hợp?
Category	Read/Create/Update/Delete/Search.	Field nào bắt buộc? Status biểu diễn thế nào?
News	Read/Create/Update/Delete/Search; gắn Category; status; tags nếu có.	categoryId liên hệ ra sao? createdBy là gì?
Users	Read/Create/Update/Delete/Search; role Admin/Staff.	Role 1/2 hiển thị và validate ra sao?
Settings	Cấu hình/profile đơn giản nếu đề yêu cầu.	Phạm vi tối thiểu là gì?

2.2 Phác thảo dữ liệu trước khi code

Entity	Field tối thiểu theo đề/prompt	Quy tắc cần tự quyết định
Category	id, name, status	id unique; name không rỗng; status 0/1.
News Article	id, title, content, categoryId, createdBy, status, tags (nếu có)	categoryId phải tham chiếu category tồn tại; status 0/1.
Account/User	id, username, password/mock password, role, status (nếu dùng)	role 1=Admin, 2=Staff; username nên unique.
Session/Auth	currentUser hoặc isAuthenticated	Cách lưu state/localStorage; logout reset gì.

2.3 Bảng Input – Process – Output cho CRUD

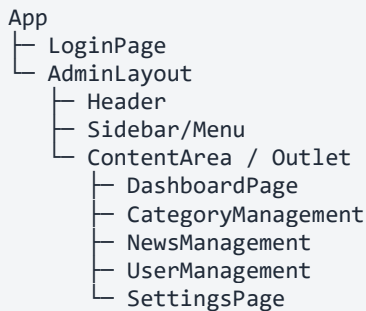
Use case	Input	Process	Output/UI
Read	Danh sách hiện có	map/render/filter status label	Table/list hoặc empty state
Create	Form rỗng → dữ liệu mới	validate → tạo id → immutable add	Record mới xuất hiện; dialog đóng
Update	Selected item + form	prefill → validate → replace by id	Đúng record được cập nhật
Delete	Selected id/item	confirm → immutable remove	Record mất khỏi list; cancel không xóa
Search	keyword (+ filter nếu có)	normalize → filter	Filtered list / no-result state

2.4 Worksheet – sinh viên phải điền trước khi bắt đầu

- Component lớn của tôi là gì?
- Dữ liệu categories/news/users khởi tạo từ đâu?
- State nào local, state nào shared, state nào persistent?
- Create/Update dùng một modal chung hay hai modal? Vì sao?
- Search lọc theo field nào cho từng entity?
- Admin page được bảo vệ bằng conditional render hay Router?
- Reload trang có cần giữ login/dữ liệu không? Nếu có, lưu ở đâu?
- Khi danh sách rỗng hoặc không có kết quả search, UI hiển thị gì?
- Validation tối thiểu cho từng form là gì?

3. HƯỚNG DẪN THIẾT KẾ GIẢI PHÁP – SOLUTION DESIGN & REASONING

3.1 Component tree gợi ý – không phải đáp án bắt buộc



Sinh viên có thể chọn cấu trúc khác, nhưng phải giải thích được trách nhiệm của component, hướng truyền props/state và lý do đặt state ở vị trí đó.

3.2 State ownership map

Loại state	Ví dụ	Nên đặt ở đâu?	Lý do
Local UI state	dialog open/close, active tab, form error	Component dùng trực tiếp	Không cần chia sẻ rộng.
Form state	name/title/content/role/status	Form/dialog component	Gắn vòng đời form.
Selected item	selectedCategory/news/user	Management page hoặc dialog owner	Phục vụ update/delete.
Shared data	categories/news/users	Parent/context/service + state	Nhiều component cần đọc/cập nhật.
Auth state	currentUser/isAuthenticated	App/context hoặc storage + state	Chi phối quyền truy cập/layout.
Persistent state	mock data nếu yêu cầu giữ sau reload	localStorage + hydrate state	Không mất dữ liệu sau refresh.

3.3 Thiết kế navigation

Cách	Ưu điểm	Hạn chế	Khi phù hợp
State navigation	Đơn giản; ít dependency	URL không phản ánh page; khó mở rộng/deep link	Khi bài chưa yêu cầu Router hoặc scope rất nhỏ
React Router	URL rõ; SPA navigation; dễ mở rộng	Cần cấu hình route/protected flow	Khi sinh viên đã học Router và muốn structure rõ

Điểm rubric: Không chấm cao chỉ vì dùng thư viện nâng cao. Điểm Solution Design đến từ quyết định phù hợp và giải thích được trade-off.

3.4 Thiết kế CRUD dialog

CREATE: click Add → mode=create → selected=null → empty form → validate → append new item → close
 UPDATE: click Edit → mode=update → selected=item → prefill form → validate → replace item by id → close
 DELETE: click Delete → selected=item → confirm → if accepted remove by id

3.5 Thiết kế search

- Giữ nguyên source list; không ghi đè source list bằng filtered result.
- Normalize keyword (trim/lowercase) trước khi so sánh nếu phù hợp.
- Category: search theo name; News: title (và có thể content); User: username.
- Định nghĩa rõ search kết hợp filter hay chỉ keyword.
- Có no-result state và nút clear nếu UI cho phép.

3.6 Design decision record – tối thiểu 6 quyết định

Quyết định	Lựa chọn của tôi	Lý do	Ảnh hưởng nếu đổi yêu cầu
Navigation			
State ownership			
Mock data persistence			
CRUD dialog reuse			
Search strategy			
Folder structure			

4. HƯỚNG DẪN TRIỂN KHAI THEO MILESTONE – IMPLEMENTATION

Không có source hoàn chỉnh: Mỗi milestone bên dưới chỉ chỉ ra cách làm, checkpoint và câu hỏi. Sinh viên phải tự viết code và chịu trách nhiệm với quyết định triển khai.

Bước 1. Tạo project Vite đúng quy ước

Mục tiêu: Có project React chạy được và tên folder StudentName_ClassCode.

Thao tác cần thực hiện

- Tạo project Vite React theo công cụ lớp yêu cầu.
- cd vào thư mục; cài dependencies; chạy development server.
- Mở package.json và xác định script dev/build.
- Tạo Git repository nếu lớp yêu cầu.

Kết quả mong đợi

- Trang mặc định chạy trên localhost.
- Tên project đúng quy ước.

Cách kiểm tra

- Chạy npm run dev.
- Tắt server rồi chạy lại để chắc chắn môi trường tái lập.

Lỗi thường gặp / dấu hiệu cần xem lại

- Tên project sai; tạo nhầm template; thiếu npm install; chạy lệnh ở sai thư mục.

Tự kiểm tra trước khi sang bước tiếp theo

- package.json dùng để làm gì?
- npm install và npm run dev khác nhau gì?
- node_modules có nộp không? Vì sao?

Bước 2. Phân tích HTML/CSS template

Mục tiêu: Biết phần nào giữ nguyên, phần nào chuyển thành React component.

Thao tác cần thực hiện

- Mở template và đánh dấu Header/Sidebar/Content/Footer.
- Liệt kê CSS/JS/assets đi kèm.
- Xác định asset nào đặt trong public/assets hoặc src/assets.
- Không copy toàn bộ HTML vào một component duy nhất.

Kết quả mong đợi

- Có sơ đồ layout và danh sách component dự kiến.

Cách kiểm tra

- Đối chiếu template với component tree.
- Chỉ ra 3 phần có thể tái sử dụng.

Lỗi thường gặp / dấu hiệu cần xem lại

- Đường dẫn asset sai; nhúng script DOM cũ gây xung đột; giữ inline onclick thay vì React event.

Tự kiểm tra trước khi sang bước tiếp theo

- Phần nào là layout?
- CSS global và component CSS khác gì?
- DOM script cũ nào cần loại bỏ?

Bước 3. Tạo Login Page và mock authentication

Mục tiêu: Login Admin/Admin hoạt động; credential sai hiển thị lỗi.

Thao tác cần thực hiện

- Tạo form username/password bằng controlled inputs.
- Validate empty trước khi kiểm tra credential.
- Khi Admin/Admin đúng: cập nhật auth state và chuyển admin view.
- Khi sai: hiển thị thông báo; không vào admin view.
- Thiết kế logout/reset session.

Kết quả mong đợi

- Login đúng vào admin page; login sai không vào được.
- Có thông báo lỗi dễ hiểu.

Cách kiểm tra

- Test empty, wrong user, wrong password, correct Admin/Admin, logout.

Lỗi thường gặp / dấu hiệu cần xem lại

- So sánh sai hoa/thường không chú ý; auth state không reset; lưu password thật không cần thiết; login logic rải rác.

Tự kiểm tra trước khi sang bước tiếp theo

- Controlled input là gì?
- State nào đổi khi login thành công?
- Nếu reload, auth state còn không? Vì sao?

Bước 4. Tạo Header, logo AI và Admin Layout

Mục tiêu: Có master layout dùng chung cho các trang quản trị.

Thao tác cần thực hiện

- Tạo logo bằng công cụ AI theo quy định; lưu asset.
- Tách Header và Sidebar/Menu.
- Tạo content region cho page hiện tại.
- Hiển thị tên hệ thống/current user nếu phù hợp.

Kết quả mong đợi

- Header/logo/menu hiển thị ổn; layout bám template.

Cách kiểm tra

- Resize cửa sổ cơ bản; kiểm tra asset; kiểm tra menu không che content.

Lỗi thường gặp / dấu hiệu cần xem lại

- Asset 404; CSS template áp sai scope; một page tự tạo lại Header làm lặp code.

Tự kiểm tra trước khi sang bước tiếp theo

- Props nào Header cần?
- Vì sao Header/Sidebar nên tái sử dụng?

Bước 5. Tạo menu và navigation

Mục tiêu: Dashboard/Category/News/Users/Settings chuyển đúng view.

Thao tác cần thực hiện

- Chọn state navigation hoặc React Router.
- Tạo cấu hình menu rõ ràng.
- Tô active menu.
- Nếu có auth guard, thử truy cập admin khi chưa login.

Kết quả mong đợi

- Bấm menu hiển thị đúng page; không full reload khi SPA.

Cách kiểm tra

- Click toàn bộ menu; back/forward nếu dùng Router; logout rồi thử admin.

Lỗi thường gặp / dấu hiệu cần xem lại

- Dùng <a> gây reload không chủ ý; route/path sai; active state không đồng bộ.

Tự kiểm tra trước khi sang bước tiếp theo

- Vì sao chọn navigation hiện tại?
- Nếu thêm Reports page, cần sửa ở đâu?

Bước 6. Tạo mock data và service/data layer

Mục tiêu: Có dữ liệu hợp lý cho Category, News, Users và đường thay thế API sau này.

Thao tác cần thực hiện

- Tạo data mẫu có id duy nhất, status/role hợp lệ.
- Tách data/service khỏi UI nếu có thể.
- Định nghĩa thao tác get/create/update/delete ở mức kiến trúc.
- Nếu dùng localStorage, định nghĩa quy trình load/save rõ ràng.

Kết quả mong đợi

- Các trang đọc được data; relation categoryId hợp lý.

Cách kiểm tra

- Kiểm tra id unique; News trở category tồn tại; role/status đúng miền giá trị.

Lỗi thường gặp / dấu hiệu cần xem lại

- Mutate mảng gốc; id trùng; categoryId không tồn tại; parse localStorage không kiểm tra null.

Tự kiểm tra trước khi sang bước tiếp theo

- Vì sao tách service?
- Sau này thay bằng Spring Boot API thì thay tầng nào?

Bước 7. Hoàn thiện READ cho 3 khu vực quản lý

Mục tiêu: Category, News, Users hiển thị dữ liệu có empty state.

Thao tác cần thực hiện

- Render list/table bằng map.
- Dùng key ổn định.
- Chuyển role/status số thành nhãn dễ hiểu.
- Thiết kế empty state.

Kết quả mong đợi

- Bảng/list hiển thị đúng record và field.

Cách kiểm tra

- Test có dữ liệu và mảng rỗng; kiểm tra console warning key.

Lỗi thường gặp / dấu hiệu cần xem lại

- Dùng index làm key không cần thiết; lỗi field name; status hiển thị 0/1 khó đọc.

Tự kiểm tra trước khi sang bước tiếp theo

- Vì sao cần key?
- UI re-render khi data state đổi như thế nào?

Bước 8. Hoàn thiện CREATE/UPDATE bằng popup dialog

Mục tiêu: Create/Update đúng yêu cầu và dùng dialog.

Thao tác cần thực hiện

- Thiết kế mode create/update hoặc tách hai flow rõ.
- Create mở form rỗng; Update prefill selected item.
- Validate field bắt buộc.
- Submit dùng immutable update.
- Reset/close dialog đúng lúc.

Kết quả mong đợi

- Create thêm đúng record; Update đúng item; invalid input không cập nhật.

Cách kiểm tra

- Test Create hợp lệ/thiếu field; Update từng item; Cancel; mở Edit liên tiếp 2 record.

Lỗi thường gặp / dấu hiệu cần xem lại

- State cũ còn trong Create; update nhầm id; mutate selected object; đóng modal trước khi validate.

Tự kiểm tra trước khi sang bước tiếp theo

- Create và Update khác nhau ở state nào?
- Vì sao không mutate trực tiếp array/object?

Bước 9. Hoàn thiện DELETE với confirmation

Mục tiêu: Chỉ xóa sau khi user xác nhận.

Thao tác cần thực hiện

- Lưu selected id/item.
- Hiển thị confirmation prompt/dialog.
- Confirm → immutable remove; Cancel → không đổi data.
- Nếu entity có relation, xác định policy đơn giản và nhất quán.

Kết quả mong đợi

- Confirm xóa đúng item; cancel giữ nguyên.

Cách kiểm tra

- Test first/middle/last/only record; cancel; xóa item vừa tạo.

Lỗi thường gặp / dấu hiệu cần xem lại

- Xóa nhầm do index; không reset selected item; UI chưa cập nhật; relation trở nên invalid.

Tự kiểm tra trước khi sang bước tiếp theo

- filter và splice khác nhau gì với React state?
- Nếu xóa Category đang được News dùng thì bạn xử lý thế nào?

Bước 10. Hoàn thiện SEARCH

Mục tiêu: Search không phá source data và hoạt động cho các entity yêu cầu.

Thao tác cần thực hiện

- Tạo keyword state riêng từng page hoặc logic tái sử dụng.
- Lọc theo field phù hợp; normalize keyword.

- Hiển thị no-result state; hỗ trợ clear search nếu phù hợp.
- Đảm bảo CRUD sau search vẫn cập nhật source list đúng.

Kết quả mong đợi

- Keyword thay đổi → result đúng; clear → full list.

Cách kiểm tra

- Test chữ hoa/thường, khoảng trắng, no result, create/update trong khi đang search.

Lỗi thường gặp / dấu hiệu cần xem lại

- Ghi đè source list; search trên filtered list cũ; null field gây lỗi toLowerCase.

Tự kiểm tra trước khi sang bước tiếp theo

- Source data và displayed data khác nhau như thế nào?
- Nếu có API thật, search sẽ thay đổi ra sao?

Bước 11. Hoàn thiện UI/UX, README và evidence

Mục tiêu: Project ở trạng thái có thể nộp và có thể bảo vệ.

Thao tác cần thực hiện

- Kiểm tra responsive cơ bản, label/button/dialog consistency.
- Loại console.log thừa nhưng giữ debug evidence riêng nếu cần.
- Viết README với setup/run/test credential/features/limitations/AI usage.
- Tạo screenshot/evidence và test matrix.
- Commit/tag phiên bản nộp nếu dùng Git.

Kết quả mong đợi

- Người khác clone/copy source, npm install và chạy được theo README.

Cách kiểm tra

- Thử chạy từ terminal mới; đọc README như người chưa biết project; chạy full smoke test.

Lỗi thường gặp / dấu hiệu cần xem lại

- Thiếu README; dependencies không khớp; hard-coded path máy cá nhân; screenshot không chứng minh chức năng.

Tự kiểm tra trước khi sang bước tiếp theo

- Một người khác cần mấy bước để chạy project?
- Giới hạn nào của bài cần nêu rõ?

4.12 Bảng milestone và bằng chứng

Milestone	Gate đạt	Evidence nên lưu
M1	Project chạy	Screenshot terminal + UI
M2	Template decomposition	Component sketch
M3	Login	3–5 login tests
M4	Layout	Admin screenshot
M5	Navigation	Menu demo
M6	Data/service	Data model note
M7	Read	3 management lists
M8	Create/Update	Modal tests
M9	Delete	Confirm/cancel evidence
M10	Search	Search cases
M11	Release candidate	README + full test matrix + Git tag/zip

5. HƯỚNG DẪN KIỂM THỬ – TESTING & VERIFICATION

Rubric không chỉ hỏi “chạy được chưa?” mà còn đánh giá chất lượng test. Sinh viên nên chuẩn bị test theo 3 lớp: normal case, negative case và edge case.

5.1 Test Matrix bắt buộc

#	Nhóm	Case	Thao tác	Kết quả mong đợi	Actual/Pass
1	Login	Empty	Submit khi username/password rỗng	Không login; hiển thị validation	
2	Login	Wrong credential	Nhập sai username hoặc password	Không vào admin page	
3	Login	Valid Admin	Admin/Admin	Vào admin page	
4	Auth	Logout	Logout sau khi login	Về login; admin không còn truy cập được	
5	Layout	Menu completeness	Quan sát sidebar/menu	Có đủ 5 mục theo đề	
6	Navigation	All menus	Bấm từng menu	Đúng nội dung; không full reload nếu SPA	
7	Category	Read	Mở Category Management	Danh sách hiển thị đúng	
8	Category	Create valid	Mở Add dialog, nhập hợp lệ, submit	Record mới xuất hiện	
9	Category	Create invalid	Bỏ trống field bắt buộc	Không thêm; có feedback	
10	Category	Update	Edit record, đổi field, save	Đúng record được cập nhật	
11	Category	Delete cancel	Delete → Cancel	Record vẫn còn	
12	Category	Delete confirm	Delete → Confirm	Đúng record bị xóa	
13	Category	Search hit	Nhập keyword tồn tại	Danh sách lọc đúng	
14	Category	Search miss	Nhập keyword không tồn tại	No-result state	
15	News	Read	Mở News Management	Danh sách hiển thị đúng	
16	News	Create valid	Mở Add dialog, nhập hợp lệ, submit	Record mới xuất hiện	
17	News	Create invalid	Bỏ trống field bắt buộc	Không thêm; có feedback	
18	News	Update	Edit record, đổi field, save	Đúng record được cập nhật	
19	News	Delete cancel	Delete → Cancel	Record vẫn còn	
20	News	Delete confirm	Delete → Confirm	Đúng record bị xóa	
21	News	Search hit	Nhập keyword tồn tại	Danh sách lọc đúng	
22	News	Search miss	Nhập keyword không tồn tại	No-result state	
23	Users	Read	Mở Users Management	Danh sách hiển thị đúng	
24	Users	Create valid	Mở Add dialog, nhập hợp lệ, submit	Record mới xuất hiện	
25	Users	Create invalid	Bỏ trống field bắt buộc	Không thêm; có feedback	
26	Users	Update	Edit record, đổi field, save	Đúng record được cập nhật	
27	Users	Delete cancel	Delete → Cancel	Record vẫn còn	
28	Users	Delete confirm	Delete → Confirm	Đúng record bị xóa	

#	Nhóm	Case	Thao tác	Kết quả mong đợi	Actual/Pass
29	Users	Search hit	Nhập keyword tồn tại	Danh sách lọc đúng	
30	Users	Search miss	Nhập keyword không tồn tại	No-result state	
31	News	Relation	Create/Update news chọn category	categoryId hợp lệ	
32	User	Role	Hiển thị role 1/2	Hiển thị Admin/Staff hợp lý	
33	Status	Display	Record status 0/1	Hiển thị Inactive/Active	
34	Data	Empty state	Tạo mảng rỗng hoặc xóa hết	UI không crash; có thông báo	
35	Persistence	Reload	Reload nếu dùng localStorage	Dữ liệu/session theo đúng design decision	

5.2 Test quality – cách đạt điểm cao

- Không chỉ test happy path; phải có invalid form, cancel delete, empty list và no-result search.
- Expected result phải cụ thể, có thể quan sát; tránh ghi “hoạt động đúng”.
- Nếu bug được phát hiện, ghi defect → fix → retest.
- Nếu dùng localStorage, test cả first-run khi storage chưa có và reload sau CRUD.
- Nếu dùng Router, test direct URL/back-forward nếu nằm trong design.

5.3 Evidence package

Evidence	Nội dung	Dùng để chứng minh
E01	Screenshot login success/error	Problem understanding + implementation
E02	CRUD screenshots hoặc demo trực tiếp	Implementation
E03	Test matrix có Actual/Pass	Testing
E04	Debug log 2–3 lỗi thật	Debugging
E05	Component/data/state sketch	Design
E06	AI log nếu có	AI Usage
E07	Git history (nếu dùng)	Engineering process
E08	README	Reproducibility

6. HƯỚNG DẪN DEBUG – DEBUGGING & PROBLEM SOLVING

6.1 Quy trình debug 5 bước

1. Mô tả triệu chứng bằng hành vi quan sát được, không đoán nguyên nhân ngay.
2. Khoanh vùng component/state/event liên quan.
3. Kiểm tra dữ liệu vào: props, state, selected item, form values.
4. Kiểm tra điểm biến đổi: event handler, filter/map, setter, navigation.
5. Sửa thay đổi nhỏ nhất → chạy lại đúng test case → kiểm tra regression.

6.2 Debug decision table

Triệu chứng	Điểm kiểm tra đầu tiên	Câu hỏi dẫn đường
Component không hiển thị	Import/export + console	Tên file/path đúng? Component có return JSX?
List rỗng bất thường	Data/state trước map	Data có thật? Field name đúng?
Modal không mở	show + onClick + props	Event có chạy? show có đổi?
Edit sai item	selected id + update condition	Dùng id hay index? selected có đúng?
Delete không cập nhật UI	new array + setter	Có mutate trực tiếp? setter được gọi?

Triệu chứng	Điểm kiểm tra đầu tiên	Câu hỏi dẫn đường
Search sai	keyword + source list	Có ghi đề source data? normalize chưa?
Login không vào admin	credential + auth state	Condition đúng? navigation/conditional render đúng?
Reload mất dữ liệu	storage strategy	Design có yêu cầu persistence? hydrate/save ở đâu?

6.3 Debug log mẫu để sinh viên tự ghi

Lỗi	Giả thuyết	Cách kiểm tra	Root cause	Fix	Retest

Trong buổi chấm: Nếu giảng viên đưa ra một bug mới, hãy nói thành tiếng: triệu chứng → giả thuyết → nơi kiểm tra → bằng chứng → sửa. Đây là cách chứng minh Debugging & Problem Solving.

7. HƯỚNG DẪN CHUẨN BỊ GIẢI THÍCH CODE – CODE EXPLANATION

Sinh viên phải mở được code của mình và giải thích theo luồng thực thi. Không nên trả lời bằng định nghĩa thuộc lòng nếu không chỉ được vị trí code tương ứng.

7.1 Khung trả lời 5 tầng

Tầng	Cách giải thích
1. Purpose	File/component này giải quyết trách nhiệm gì?
2. Input	Props/state/data/service nào đi vào?
3. Trigger	Event/useEffect/navigation nào kích hoạt?
4. Transform	Logic nào validate/filter/update dữ liệu?
5. Output	State/UI/navigation nào thay đổi và React render lại ra sao?

7.2 Các chủ đề phải giải thích được

- Project structure và trách nhiệm các thư mục/component.
- JSX, component, props, state; props khác state.
- Re-render: điều gì làm state đổi và UI cập nhật.
- map + key; conditional rendering; empty/no-result state.
- onClick/onChange/onSubmit; preventDefault.
- Controlled form và validation.
- Immutable update cho create/update/delete.
- Modal state: show/mode/selected item/form data.
- CRUD flow và Search flow.
- Mock authentication/login/logout.
- Navigation nếu dùng state hoặc Router.
- localStorage/service layer nếu có.

7.3 “Mở file và chỉ code” – rehearsal checklist

Yêu cầu rehearsal	Tôi có thể làm trong ≤ 30 giây?
Mở file xử lý login và chỉ condition Admin/Admin	☐

Yêu cầu rehearsal	Tôi có thể làm trong ≤ 30 giây?
Mở nơi lưu auth state / logout	☐
Mở Category create/update logic	☐
Mở News relation với categoryId	☐
Mở User role mapping	☐
Mở delete confirmation logic	☐
Mở search logic và chỉ source list vs displayed list	☐
Mở modal state và giải thích create/update mode	☐
Mở nơi lưu/khởi tạo localStorage nếu có	☐
Mở route/menu config nếu dùng Router	☐

8. HƯỚNG DẪN XỬ LÝ YÊU CẦU THAY ĐỔI – MODIFICATION / TRANSFER TASK

Mục tiêu: chứng minh sinh viên không chỉ nhớ code hiện tại mà có thể chuyển kiến thức sang yêu cầu mới. Khi luyện, không dùng AI trong lúc thực hiện modification nếu mô phỏng buổi đánh giá trực tiếp.

8.1 Bài luyện mức cơ bản

- Thêm nút Reset trong Create/Update dialog.
- Thêm validation email cho User.
- Thêm field status cho News nếu hiện chưa có UI.
- Thêm filter Active/Inactive.
- Thêm success message sau Create/Update/Delete.

8.2 Bài luyện mức trung bình

- Thêm role Staff: đăng nhập được; không thấy Users; chỉ truy cập News/Category theo yêu cầu giả định.
- Thêm pagination cho News.
- Thêm sort theo title hoặc createdAt.
- Kết hợp search + category filter.
- Cảnh báo khi đóng dialog có form đang thay đổi.

8.3 Bài luyện nâng cao

- Thay mock array bằng json-server/mock API.
- Tách service layer để sẵn sàng gọi Spring Boot.
- Thêm tags cho News và cập nhật form/list/search.
- Dashboard thống kê số News/Category/User và active/inactive.
- Dark mode có persistent preference.

8.4 Impact Analysis Worksheet

Change request	Component ảnh hưởng	State/data ảnh hưởng	UI ảnh hưởng	Test mới/cũ phải sửa

Cách trả lời tốt: Trước khi code modification, hãy nói ngắn: “Yêu cầu này ảnh hưởng data model → form → list rendering → validation → test cases”. Sau đó mới sửa.

9. HƯỚNG DẪN SỬ DỤNG AI CÓ KIỂM SOÁT – AI USAGE & ENGINEERING PRACTICE

9.1 AI có thể hỗ trợ

- Giải thích concept hoặc một đoạn code do sinh viên đã viết.
- Gợi ý component structure hoặc state model.
- Đưa giả thuyết debug và checklist kiểm tra.
- Sinh test case/edge case để sinh viên chọn và kiểm chứng.
- Review naming, accessibility, form validation, README outline.
- Tạo prompt hình ảnh/logo cho FUNewsManagementSystem.

9.2 AI không được thay thế năng lực

- Không yêu cầu AI tạo toàn bộ assignment để nộp.
- Không copy source trọn bộ mà không hiểu.
- Không dùng AI để che giấu việc không giải thích được state/event/data flow.
- Không coi output AI là đúng nếu chưa chạy/test/đối chiếu yêu cầu.

9.3 Quy trình AI → Verify → Adopt

- 1) Ask: đặt câu hỏi cụ thể
- 2) Inspect: đọc và xác định giả định của AI
- 3) Adapt: chỉnh cho project của mình
- 4) Run: chạy đúng test case
- 5) Explain: tự giải thích được logic
- 6) Record: ghi prompt + thay đổi + cách kiểm chứng

9.4 AI usage log

#	Prompt	Mục đích	Gợi ý AI	Tôi đã kiểm chứng thế nào?	Tôi đã sửa gì?

10. RUBRIC ĐÁNH GIÁ – 10.0 ĐIỂM

Rubric đánh giá năng lực thực chất, không chỉ đánh giá việc sản phẩm “có chạy”. Mỗi tiêu chí có dấu hiệu quan sát được qua sản phẩm, demo, interview, test/debug và modification task.

#	Tiêu chí	Điểm tối đa	Bằng chứng chính
1	Problem Understanding	1.0	Requirement map + trả lời constraints/input/output
2	Solution Design & Reasoning	1.5	Component/data/state design + design decisions
3	Implementation	2.0	Project chạy + core functions + code structure
4	Testing & Verification	1.5	Test matrix + edge case + evidence
5	Debugging & Problem Solving	1.5	Phân tích/sửa lỗi + debug log/live task
6	Code Explanation	1.0	Interview + mở code và giải thích flow
7	Modification / Transfer Task	1.0	Thay đổi requirement nhỏ và kiểm thử lại

#	Tiêu chí	Điểm tối đa	Bằng chứng chính
8	AI Usage & Engineering Practice	0.5	AI log + verification + không phụ thuộc AI

Tiêu chí	Excellent / Tối đa	Good / Khá	Fair / Trung bình	Poor / Chưa đạt
1. Problem Understanding – 1.0	0.9–1.0: Nêu đúng toàn bộ scope; chỉ ra màn hình/entity/CRUD+Search/constraints; giải thích input-output và relation.	0.7–0.8: Hiểu phần lớn; thiếu 1–2 chi tiết nhỏ nhưng không sai core requirement.	0.4–0.6: Hiểu rời rạc; bỏ sót constraint quan trọng hoặc không rõ input/output.	0–0.3: Không mô tả được bài toán; nhầm core requirement hoặc chỉ dựa vào code đã có.
2. Solution Design & Reasoning – 1.5	1.35–1.5: Component/data/state/auth/navigation thiết kế hợp lý; nêu trade-off và impact khi thay đổi.	1.05–1.30: Thiết kế hợp lý, giải thích được phần chính; một số quyết định còn theo thói quen.	0.60–1.00: Cấu trúc chạy được nhưng state/component placement chưa rõ; khó giải thích design decision.	0–0.55: Thiết kế chấp vá; logic rời rạc; không biết vì sao đặt code/state như vậy.
3. Implementation – 2.0	1.8–2.0: Project chạy ổn; login/layout/menu/CRUD+Search/dialog/confirm đầy đủ; code rõ, state update đúng.	1.4–1.75: Core functions hầu hết chạy; có lỗi nhỏ hoặc code quality chưa đều.	0.8–1.35: Chạy một phần; thiếu/mất ổn định ở một số CRUD/Search/dialog; code khó bảo trì.	0–0.75: Project không chạy hoặc thiếu nhiều core requirements; sao chép/không làm chủ code.
4. Testing & Verification – 1.5	1.35–1.5: Test matrix đầy đủ normal/negative/edge; có expected/actual/evidence; retest bug fix.	1.05–1.30: Test core functions và một số edge case; evidence tương đối rõ.	0.60–1.00: Test chủ yếu happy path; thiếu edge case/actual evidence.	0–0.55: Hầu như không có test case có cấu trúc; chỉ demo ngẫu nhiên.
5. Debugging & Problem Solving – 1.5	1.35–1.5: Khoanh vùng có hệ thống, dùng DevTools/log/debugger hợp lý, tìm root cause và retest.	1.05–1.30: Sửa được lỗi phổ biến, giải thích nguyên nhân tương đối rõ.	0.60–1.00: Thử-sai nhiều; cần gợi ý; sửa được nhưng chưa rõ root cause.	0–0.55: Không biết bắt đầu debug; phụ thuộc người khác/AI để sửa.
6. Code Explanation – 1.0	0.9–1.0: Mở đúng file và giải thích rõ purpose-input-trigger-transform-output; hiểu state/props/event/CRUD flow.	0.7–0.8: Giải thích được phần lớn code mình viết; còn một số chi tiết chưa sâu.	0.4–0.6: Giải thích cú pháp cơ bản nhưng không theo được luồng dữ liệu/thực thi.	0–0.3: Không giải thích được code quan trọng hoặc câu trả lời mâu thuẫn với source.
7. Modification / Transfer – 1.0	0.9–1.0: Phân tích impact, sửa requirement nhỏ đúng, test lại và giải thích thay đổi.	0.7–0.8: Thực hiện được thay đổi với ít gợi ý; test cơ bản.	0.4–0.6: Cần nhiều gợi ý; sửa được một phần hoặc gây regression.	0–0.3: Không thể sửa yêu cầu nhỏ nếu không có mẫu sẵn.
8. AI Usage & Engineering Practice – 0.5	0.45–0.5: Dùng AI minh bạch, prompt có mục tiêu, tự kiểm chứng/chỉnh sửa, giải thích được kết quả.	0.35–0.4: Dùng AI hợp lệ và có kiểm chứng nhưng log/giải trình chưa đầy đủ.	0.20–0.30: Có dùng AI nhưng kiểm chứng hạn chế; phụ thuộc gợi ý khá nhiều.	0–0.15: Không minh bạch hoặc phụ thuộc AI/source trọn bộ mà không hiểu.

10.1 Cách tính điểm

Rubric Score = tổng điểm 8 tiêu chí, tối đa 10.0. Nếu lớp áp dụng hệ số nộp trễ đã công bố: Final Score = Rubric Score × Timeliness Coefficient.

Không dùng “một lỗi = mất toàn bộ điểm” trừ khi quy định chính thức nói vậy: Rubric nên phản ánh mức năng lực quan sát được ở từng tiêu chí. Ví dụ sản phẩm chạy nhưng không giải thích được code vẫn có thể có Implementation tương đối, nhưng Code Explanation/Modification/AI Usage sẽ thấp.

10.2 Scorecard dành cho sinh viên tự chấm trước buổi đánh giá

Tiêu chí	Max	Tự chấm	Evidence tôi sẽ mở
Problem Understanding	1.0		
Solution Design & Reasoning	1.5		
Implementation	2.0		
Testing & Verification	1.5		
Debugging & Problem Solving	1.5		
Code Explanation	1.0		
Modification / Transfer	1.0		
AI Usage & Engineering Practice	0.5		
Tổng	10.0		

11. CHECKLIST TRƯỚC KHI NỘP BÀI VÀ TRƯỚC BUỔI ĐÁNH GIÁ

11.1 Checklist yêu cầu bắt buộc

- ☐ Project Vite + ReactJS; tên StudentName_ClassCode.
- ☐ Login page; Admin/Admin vào admin page.
- ☐ Header và logo tạo bằng AI.
- ☐ Menu đủ Dashboard, Category, News, Users, Settings.
- ☐ CRUD + Search cho Category, News, Users/Account.
- ☐ Create/Update dùng dialog.
- ☐ Delete có confirmation.
- ☐ README có cách chạy và credential test.

11.2 Checklist chất lượng kỹ thuật

- ☐ npm run dev chạy không lỗi.
- ☐ Không có console error nghiêm trọng.
- ☐ Component tách hợp lý.
- ☐ Không mutate trực tiếp state.
- ☐ Có empty/no-result state.
- ☐ Form có validation tối thiểu.
- ☐ Mock data có id, role/status/category relation hợp lý.
- ☐ Có test matrix và đã chạy smoke test cuối.

11.3 Checklist sẵn sàng phòng vấn

- ☐ Tôi giải thích được lệnh tạo/chạy project và package.json.
- ☐ Tôi mở đúng file login trong 30 giây.
- ☐ Tôi giải thích được CRUD flow cho ít nhất một entity.
- ☐ Tôi giải thích được selected item + modal mode.
- ☐ Tôi giải thích được search mà không đọc thuộc lòng.
- ☐ Tôi biết vì sao không mutate state.
- ☐ Tôi biết dữ liệu mất/còn gì sau reload.
- ☐ Tôi có thể phân tích một bug bằng state/props/event flow.
- ☐ Tôi đã luyện ít nhất một modification task.

11.4 Checklist hồ sơ đánh giá

- ☐ Source/zip hoặc repo đúng phiên bản nộp.
- ☐ README final.
- ☐ Test matrix final.
- ☐ 2–3 debug log.
- ☐ Component/data/state sketch.
- ☐ AI usage log nếu có/yêu cầu.
- ☐ Không còn file secret hoặc asset path theo máy cá nhân.

11.5 Kế hoạch chuẩn bị 60 phút trước buổi đánh giá

Thời gian	Việc làm
0–10 phút	Clone/mở bản nộp sạch; npm install; npm run dev; xác nhận đúng version.
10–25 phút	Chạy smoke test: login → menu → mỗi CRUD → search → delete cancel/confirm.
25–35 phút	Mở component tree/data/state design; luyện giải thích 3 flow: login, CRUD, search.
35–45 phút	Luyện 1 debug case và 1 modification case không dùng AI.
45–55 phút	Kiểm tra README/test matrix/AI log/evidence; đóng tab/file không cần thiết.
55–60 phút	Mở sẵn project + terminal; đảm bảo demo bắt đầu từ trạng thái ổn định.

12. PHỤ LỤC – CÂU HỎI INTERVIEW, LỆNH VÀ QUY LUẬT THỰC THI

12.1 Lệnh và môi trường

Câu hỏi	Điểm cần trả lời
Lệnh tạo project Vite React?	Nêu lệnh/cách theo môi trường lớp và giải thích template React.
npm install làm gì?	Đọc package.json/package-lock và cài dependencies vào node_modules.
npm run dev làm gì?	Chạy script dev trong package.json, khởi động dev server.
package.json chứa gì?	Metadata, scripts, dependencies/devDependencies.
node_modules là gì?	Dependencies cài cục bộ; thường không commit/nộp vì có thể tái tạo.
Dừng dev server?	Ctrl+C tại terminal đang chạy.
Port bạn xử lý?	Dừng process cũ hoặc chạy port khác theo công cụ.

12.2 React fundamentals

- Component là gì?
- JSX là gì?
- Props là gì? State là gì? Khác nhau thế nào?
- Khi nào component re-render?
- Vì sao không mutate trực tiếp state?
- Controlled input là gì?
- Vì sao map cần key?
- Conditional rendering là gì?
- preventDefault trong form dùng để làm gì?

12.3 Luồng thực thi

- Khi mở ứng dụng, component nào quyết định Login hay Admin?
- Khi submit Admin/Admin, các bước state/event/render xảy ra thế nào?
- Khi bấm menu, component nào thay đổi?
- Khi bấm Add, state nào đổi để modal mở?
- Create: dữ liệu form đi từ input đến danh sách như thế nào?
- Update: selected item được prefill và thay thế theo id ra sao?
- Delete: confirmation bảo vệ thao tác như thế nào?
- Search: source list và displayed list khác nhau ra sao?
- Reload: state nào mất, dữ liệu nào có thể còn nếu dùng localStorage?

12.4 Design & architecture

- Vì sao tách Header/Sidebar/Content?
- Vì sao đặt state danh sách ở component hiện tại?
- Vì sao dùng dialog cho Create/Update?
- Vì sao dùng mock data và giới hạn của nó?
- Nếu kết nối Spring Boot API, tầng nào cần đổi?
- Nếu thêm Staff role, cần sửa auth/menu/page nào?
- Nếu thêm pagination/search server-side, kiến trúc đổi gì?

12.5 Debugging interview

- List không hiển thị: bạn kiểm tra gì đầu tiên?
- Modal không mở: kiểm tra state/event/props thế nào?
- Delete không re-render: dấu hiệu mutate state là gì?
- Login đúng nhưng không vào admin: khoanh vùng ở đâu?
- Search sai: kiểm tra keyword/source data/field nào?

- Reload mất dữ liệu: đó là bug hay đúng design?

12.6 AI usage interview

- Bạn dùng AI vào phần nào?
- Prompt cụ thể nào hữu ích nhất?
- Bạn đã kiểm chứng output bằng test nào?
- Bạn đã thay đổi gì so với gợi ý AI?
- Nếu AI đưa code sai hoặc dùng API không tồn tại, bạn phát hiện thế nào?
- Phần nào của project hoàn toàn do bạn tự thiết kế?

12.7 Mini interview cards – tự luyện

Card	Yêu cầu 2–4 phút
A – Login	Mở login code; giải thích controlled inputs, validation, Admin/Admin và state sau success.
B – Create/Update	Mở một CRUD dialog; chỉ show/mode/selected/form; giải thích immutable update.
C – Delete	Giải thích selected id, confirmation, cancel/confirm và re-render.
D – Search	Giải thích keyword → filter → displayed list; chứng minh source list không bị mất.
E – Debug	Giảng viên giả định “Edit cập nhật nhầm record”; nêu cách khoanh vùng theo id/selected/update logic.
F – Transfer	Thêm validation email hoặc filter status; nêu impact trước khi sửa.

12.8 Final self-declaration

Sinh viên tự xác nhận trước khi nộp: “Tôi đã tự kiểm tra project, có thể giải thích các phần chính, có thể thực hiện một thay đổi nhỏ và chịu trách nhiệm với mọi code/tài nguyên/AI output tôi sử dụng.”

KẾT LUẬN

Assignment 01 không nên được xem là bài “làm UI cho chạy”. Muốn đạt điểm cao theo rubric, sinh viên cần chứng minh một chuỗi năng lực hoàn chỉnh: hiểu yêu cầu → thiết kế có lý do → triển khai đúng → kiểm thử → debug → giải thích → thích nghi khi thay đổi → sử dụng AI có trách nhiệm. Vì vậy, hãy chuẩn bị cả sản phẩm và bằng chứng về cách bạn xây dựng sản phẩm đó.

Checklist cuối cùng: Run được • Demo đủ core scope • Test có evidence • Giải thích được code • Debug có quy trình • Làm được modification nhỏ • AI có kiểm chứng.